

Entrega 1 — Análisis de Complejidad y Recursividad

Juan Ocampo – Cristopher Sánchez

Caso de uso: Solver de Sudoku.

1. Definición del problema

a. Descripción del caso de uso:

El problema seleccionado es la resolución automática de un tablero de Sudoku 9x9. Un Sudoku es un tablero de 81 celdas dividido en 9 filas, 9 columnas y 9 cajas (bloques) de 3x3. Algunas celdas ya contienen un número del 1 al 9 (pistas iniciales) y el resto están vacías. El objetivo del algoritmo es asignar un número del 1 al 9 a cada celda vacía de modo que ningún número se repita en ninguna fila, columna o caja.

b. Entradas, salidas y restricciones

- **Entrada:** una matriz de 9x9 con enteros entre 0 y 9, donde 0 representa una celda vacía y los valores del 1 al 9 representan pistas ya ubicadas en el tablero.
- **Salida:** un valor booleano que indica si el tablero tiene solución, y la matriz 9x9 completa (sin ceros) que representa una solución válida, si existe.
- **Restricciones:**
 - Cada fila debe contener los números del 1 al 9 sin repetir.
 - Cada columna debe contener los números del 1 al 9 sin repetir.
 - Cada una de las 9 cajas de 3x3 debe contener los números del 1 al 9 sin repetir.
 - Las pistas iniciales del tablero no pueden modificarse.

c. Conjunto de datos de prueba

Dado que la fuerza bruta pura resuelve el problema probando todas las combinaciones posibles para las celdas vacías (9^n combinaciones, con n = número de celdas vacías), el tamaño de entrada relevante para el análisis empírico no es el tablero completo sino el número de celdas vacías n . Se definieron tres casos de prueba a partir de un mismo tablero base ya resuelto y válido, vaciando progresivamente más celdas:

Caso	Celdas vacías (n)	Combinaciones	Justificación
Pequeño	2	81	Fuerza bruta resuelve en microsegundos
Mediano	5	59.049	Crecimiento notable pero aún manejable
Grande	7	4.782.969	Varios segundos, se evidencia el crecimiento exponencial

Un Sudoku real tiene entre 40 y 60 celdas vacías. Con $n = 60$, 9^n supera 10^{57} combinaciones, por lo que la fuerza bruta pura es matemáticamente inviable de ejecutar sobre un tablero completo; esto se retoma en la sección 3.

2. Solución por fuerza bruta

La solución implementada (`fuerza_bruta.py`) genera, con `itertools.product`, todas las combinaciones posibles de valores (1 al 9) para las n celdas vacías, sin ninguna optimización. Cada combinación se coloca completa en el tablero y solo entonces se valida si el tablero resultante cumple las reglas del Sudoku. La validación se dividió en funciones independientes para mayor claridad:

- `encontrar_celdas_vacias(tablero)` — localiza las posiciones vacías.
- `verificar_filas(tablero)` — valida que ninguna fila tenga números repetidos.
- `verificar_columnas(tablero)` — valida columnas.
- `verificar_caja(tablero, fi, ci)` — valida una caja 3x3 específica.
- `verificar_cajas(tablero)` — aplica `verificar_caja` a las 9 cajas.
- `verificar_tablero(tablero)` — combina las tres verificaciones anteriores.
- `fuerza_bruta_sudoku(tablero)` — genera y prueba cada combinación posible.

El código fuente completo, documentado y ejecutable, se entrega como archivo independiente (`fuerza_bruta.py`), junto con el script de medición empírica usado para obtener los resultados de la sección 3.

3. Análisis de complejidad

a. Complejidad temporal (Big O)

Sea n el número de celdas vacías del tablero. El algoritmo genera todas las combinaciones posibles de valores del 1 al 9 para esas n celdas mediante producto cartesiano, lo que produce exactamente 9^n combinaciones en el peor caso (cuando la solución válida es la última combinación generada, o cuando no existe solución). Por cada combinación generada, el algoritmo: (a) asigna los n valores al tablero, en $O(n)$; y (b) valida el tablero completo con “`verificar_tablero`”, que recorre 9 filas, 9 columnas y 9 cajas de tamaño fijo, es decir, un costo constante $O(81) = O(1)$, independiente de n , porque el tamaño del tablero (9x9) no cambia. Por lo tanto, el costo por combinación es $O(n) + O(1) = O(n)$, y el costo total es:

$$T(n) = 9^n \cdot O(n) = O(n \cdot 9^n)$$

Como 9^n domina completamente sobre el factor lineal n para valores grandes de n , la complejidad temporal se expresa como $O(9^n)$: exponencial respecto al número de celdas vacías. Esta es la razón por la cual la fuerza bruta pura es inviable para un Sudoku real (n entre 40 y 60).

b. Complejidad espacial

El tablero ocupa siempre espacio fijo $O(81) = O(1)$, ya que su tamaño no depende de n . La lista de celdas vacías ocupa $O(n)$. Cada combinación generada por “`itertools.product`” es una tupla de tamaño n , generada de forma perezosa (una a la vez, no todas en memoria simultáneamente), por lo que el espacio adicional por iteración es $O(n)$. En consecuencia, la complejidad espacial total del algoritmo es $O(n)$.

c. Medición empírica de tiempos

Se ejecutó “fuerza_bruta_sudoku” sobre tableros generados a partir de una solución válida conocida, vaciando progresivamente entre 1 y 7 celdas, y midiendo el tiempo de ejecución con “time.perf_counter()”. Los resultados fueron:

n (celdas vacías)	9 ⁿ (teórico)	Tiempo medido (s)
1	9	0.000070
2	81	0.000267
3	729	0.000285
4	6.561	0.014406
5	59.049	0.207002
6	531.441	0.471410
7	4.782.969	9.212688

El tiempo crece de forma consistente con el modelo teórico $O(9^n)$: entre $n=6$ y $n=7$ el tiempo se multiplica por un factor cercano a 9x (de 0.47 s a 9.21 s), confirmando empíricamente el crecimiento exponencial. Extrapolando este comportamiento, un Sudoku real con 40-60 celdas vacías requeriría un tiempo de ejecución astronómicamente mayor a la edad del universo, lo que confirma que la fuerza bruta pura no es una solución práctica para el problema completo — justificando la necesidad de técnicas más eficientes (backtracking con poda, heurísticas, programación dinámica) que se desarrollarán en las siguientes entregas.

d. Gráfica comparativa

